

Project documentation

Files

The project is implemented in several python files, each having a specific goal.

slices__image_generation.py

This file is the script used to generate 3D objects based on four basic shapes :

- Rectangular cuboid
- Cylinder
- Sphere
- Torus

From these shapes several datasets are created that are used to develop the project as well as evaluate its accuracy. The 3D objects can be aligned along the classical axes or along random ones, and be homogeneous or an heterogeneous combination of different shapes. They are then "sliced" to get a set of consecutive images.

slices__unaligned.py

This is the script used to simulate the variation of placement and orientation of the slices of objects that happens when clot photographs are retrieved. It takes a dataset previously created by the script *slices__image_generation.py*

slices__reconstruction.py

This file is the script for the reconstruction pipeline. It starts with a set of images (either clot slices or one of the data sets created by the two first scripts), applies all the steps including image registration and returns the reconstructed 3D object with the identified internal structures.

slices__pixel_count.py

This file is a script that counts, for every image of a dataset, the number of pixels of every color and stores the values in an Excel file.

We can then compare the pixel count of the dataset before and after certain steps of the reconstruction pipeline to see if it preserves the proportionality of each color.

slices__metrics.py

Once the object is reconstructed, this script will take every internal object identified and compute different metrics that are stored in an Excel file.

The computed metrics are:

- Volume
- Surface area
- Surface-area-to-volume ratio
- Dimensions of the axis-aligned bounding box
- Volume of the bounding box

slices__vtk_convert.py

This script takes the *.npy* files in which the 3D internal objects are stored and converts them into *.vtk* file that can be visualized in Paraview.

functions.py

This file accts as a library for the whole project. All the functions created are stored in it and can be called as needed in the other scripts.

Functions detailed documentation

Each function of the *function.py* file is detailed below. The functions appear in the same order than in the python file and the documentation is separated in different sections based on the main usage for each function.

Objects creation

rectangular_cuboid_vox(centre, dimensions, x, y, z, axis=np.array([0,0,1]))

Creates a voxel array representing a rectangular cuboid with a given center, size, and orientation.

Parameters:

- `centre` (`np.ndarray`): Coordinates of the cuboid center point.
- `dimensions` (`np.ndarray`): Lengths of the cuboid along its three local axes. The first dimension corresponds to the specified `axis` , while the other two correspond to the orthogonal directions.
- `x` (`np.ndarray`): Array of x-coordinate values defining the voxel grid.

- `y ( np.ndarray )`: Array of y-coordinate values defining the voxel grid.
- `z ( np.ndarray )`: Array of z-coordinate values defining the voxel grid.
- `axis ( np.ndarray , optional)`: Orientation vector defining the main axis of the cuboid. Default is `[0, 0, 1]`.

Returns:

- `cuboid( np.ndarray )`: Boolean voxel array with the same shape as the input coordinate grids. `True` values indicate voxels located inside the cuboid, and `False` values indicate voxels outside the cuboid.

cylinder_vox(centre, radius, height, x, y, z, axis=np.array([0,0,1]))

Creates a voxel array representing a cylinder with a specified center, radius, height, and orientation.

Parameters:

- `centre ( np.ndarray )`: Coordinates of the cylinder center point.
- `radius ( int )`: Radius of the cylinder.
- `height ( int )`: Height of the cylinder along its axis.
- `x ( np.ndarray )`: Array of x-coordinate values defining the voxel grid.
- `y ( np.ndarray )`: Array of y-coordinate values defining the voxel grid.
- `z ( np.ndarray )`: Array of z-coordinate values defining the voxel grid.
- `axis ( np.ndarray , optional)`: Orientation vector defining the cylinder axis. Default is `[0, 0, 1]`.

Returns:

- `cyl( np.ndarray )`: Boolean voxel array with the same shape as the input coordinate grids. `True` values indicate voxels located inside the cylinder, and `False` values indicate voxels outside the cylinder.

sphere_vox(centre, radius, x, y, z)

Creates a voxel array representing a sphere with a specified center and radius.

Parameters:

- `centre ( np.ndarray )`: Coordinates of the sphere center point.
- `radius ( int )`: Radius of the sphere.
- `x ( np.ndarray )`: Array of x-coordinate values defining the voxel grid.
- `y ( np.ndarray )`: Array of y-coordinate values defining the voxel grid.
- `z ( np.ndarray )`: Array of z-coordinate values defining the voxel grid.

Returns:

- `s( np.ndarray )`: Boolean voxel array with the same shape as the input coordinate grids. `True` values indicate voxels located inside the sphere, and `False` values indicate voxels outside the sphere.

`torus_vox(centre, centre_radius, tube_radius, x, y, z, axis=np.array([0,0,1]))`

Creates a voxel array representing a torus with a specified center, major radius, tube radius, and orientation.

Parameters:

- `centre ( np.ndarray )`: Coordinates of the torus center point.
- `centre_radius ( int )`: Distance from the torus center to the center of the tube (major radius).
- `tube_radius ( int )`: Radius of the torus tube (minor radius).
- `x ( np.ndarray )`: Array of x-coordinate values defining the voxel grid.
- `y ( np.ndarray )`: Array of y-coordinate values defining the voxel grid.
- `z ( np.ndarray )`: Array of z-coordinate values defining the voxel grid.
- `axis ( np.ndarray , optional)`: Orientation vector defining the torus axis. Default is `[0, 0, 1]`.

Returns:

- `t( np.ndarray )`: Boolean voxel array with the same shape as the input coordinate grids. `True` values indicate voxels located inside the torus tube, and `False` values indicate voxels outside the torus.

Colors operations

`hexa_to_rgb(hexa_str)`

Converts a hexadecimal color string into its corresponding RGB color representation.

Parameters:

- `hexa_str ( str )`: A string starting with `#` containing the hexadecimal color code.

Returns:

- `r ( int )`: Red channel value of the color, ranging from 0 to 255.
- `g ( int )`: Green channel value of the color, ranging from 0 to 255.
- `b ( int )`: Blue channel value of the color, ranging from 0 to 255.

rgb_to_hexa(r, g, b)

Converts an RGB color representation into a hexadecimal color string.

Parameters:

- `r ( int )`: Red channel value of the color, ranging from 0 to 255.
- `g ( int )`: Green channel value of the color, ranging from 0 to 255.
- `b ( int )`: Blue channel value of the color, ranging from 0 to 255.

Returns:

- `hexa_str ( str )`: String containing the hexadecimal color code corresponding to the RGB values, starting with `#`.

color_mix(color1, color2)

Creates a new color by averaging the RGB channel values of two input colors.

Parameters:

- `color1 ( str )`: First color expressed as a hexadecimal color code string starting with `#`.
- `color2 ( str )`: Second color expressed as a hexadecimal color code string starting with `#`.

Returns:

- `new_color ( str )`: Resulting mixed color expressed as a hexadecimal color code string starting with `#`.

Image transformation

padding(images, background_color='#ffffff', size=[None, None], mode='def')

Adds padding around images to ensure that objects remain visible after image transformations. The function supports two padding modes: a default mode that adds padding equal to half the image size on each side, and a resize mode that adds padding to reach a specified target size.

Parameters:

- `images ( list[np.ndarray] )`: List of RGB images to be padded.
- `background_color ( str , optional)`: Background color used for the padding, expressed as a hexadecimal color code string starting with `#`. Default is `'#ffffff'`.

- `size ( list[int, int] , optional)`: Target image size `[x, y]` used when `mode='resize'`. Default is `[None, None]`.
- `mode ( str , optional)`: Padding mode selection. `'def'` adds padding equal to half the image dimensions on each side. `'resize'` adds padding to reach the specified `size`. Default is `'def'`.

Returns:

- `new_images ( list[np.ndarray] )`: List of padded RGB images. Each image contains the original image content with additional padding filled with the specified background color.

translation(image, t=(None, None))

Computes the transformation matrix required to translate an image. If no translation values are provided, random translation offsets are generated based on half of the image dimensions.

Parameters:

- `image ( np.ndarray )`: Image to be translated.
- `t ( tuple[int, int] , optional)`: Tuple `(x, y)` representing the desired translation offsets along the two image axes. If both values are `None`, random translation values are generated. Default is `(None, None)`.

Returns:

- `T ( np.ndarray )`: 3×3 transformation matrix representing the translation operation.

rotation(image, theta=None)

Computes the transformation matrix required to rotate an image around its center. If no rotation angle is provided, a random angle between 0 and (2π) radians is generated.

Parameters:

- `image ( np.ndarray )`: Image to be rotated. The image dimensions are used to determine the rotation center.
- `theta ( float , optional)`: Rotation angle in radians. If `None`, a random angle between 0 and (2π) is generated. Default is `None`.

Returns:

- `T ( np.ndarray )`: 3×3 transformation matrix representing the rotation around the image center.

transform(image, T)

Applies a geometric transformation to an image using a given transformation matrix.

Parameters:

- `image ( np.ndarray )`: Image to be transformed.
- `T ( np.ndarray )`: 3×3 affine transformation matrix defining the transformation to apply to the image.

Returns:

- `transfo_image ( np.ndarray )`: Transformed image. Pixels outside the original image boundaries are filled using the edge color of the image.

Creating slices

inh_container(background, objects)

Performs an inhibition operation on a background voxel array by removing the regions occupied by other objects.

Parameters:

- `background ( np.ndarray )`: Boolean voxel array representing the background object that contains the other objects.
- `objects ( list[np.ndarray] )`: List of boolean voxel arrays representing the objects contained inside the background.

Returns:

- `background ( np.ndarray )`: Updated boolean voxel array representing the background after removing the voxels occupied by the objects.

compute_intersections(index, objects)

Computes the intersections between a selected object and all other objects in a list of voxel arrays.

Parameters:

- `index ( int )`: Index of the target object in the `objects` list for which intersections are computed.
- `objects ( list[np.ndarray] )`: List of boolean voxel arrays representing all objects in the scene.

Returns:

- `obj_indexes ( list[int] )`: List of indexes corresponding to the objects that intersect with the target object.
- `intersec ( list[np.ndarray] )`: List of boolean voxel arrays representing the intersection regions between the target object and the intersecting objects.

`color_objects(background, objects, colors, background_color='#ffffff', with_intersections=False)`

Assigns colors to a background voxel array and a collection of objects, generating a color array corresponding to each voxel. The function supports optional color mixing for voxels where objects overlap.

Parameters:

- `background ( np.ndarray )`: Boolean voxel array representing the background object. It is used to determine the voxels that should receive the background object color.
- `objects ( list[np.ndarray] )`: List of boolean voxel arrays representing the objects contained within the background.
- `colors ( list[str] )`: List of hexadecimal color strings used to color the background and objects. The first color corresponds to the background, and subsequent colors correspond to the objects.
- `background_color ( str , optional)`: Hexadecimal color string used for voxels outside the background object. The string must start with `#`. Default is `'#ffffff'`.
- `with_intersections ( bool , optional)`: If `True`, overlapping regions between objects are assigned a mixed color obtained by averaging the colors of the intersecting objects. Default is `False`.

Returns:

- `facecolor ( np.ndarray )`: Array containing the hexadecimal color value assigned to each voxel. Each voxel is colored according to whether it belongs to the background, an object, or an intersection region.

`facecolor_to_rgb(facecolor)`

Converts a 2D array of hexadecimal color strings into an RGB image matrix.

Parameters:

- `facecolor ( np.ndarray )`: 2D array of hexadecimal color strings representing the color assigned to each voxel in a slice of a 3D color array.

Returns:

- `rgbcolor ( np.ndarray )`: RGB image matrix corresponding to the input color array. Each pixel contains its red, green, and blue channel values.

slice(facecolor, axis='z')

Extracts 2D image slices from a 3D voxel color array along a specified axis. Each slice is converted from a hexadecimal color representation into an RGB image format.

Parameters:

- `facecolor ( np.ndarray )`: 3D array of hexadecimal color strings representing the color assigned to each voxel.
- `axis ( str , optional)`: Axis along which the 3D array is sliced. Possible values are `'x'` , `'y'` , or `'z'` . Default is `'z'` .

Returns:

- `images ( list[np.ndarray] )`: List of RGB image arrays corresponding to the 2D slices extracted from the 3D voxel color array.

slice_object(voxelarray, facecolor, color='#ffffff')

Creates image slices for a specific object by masking voxels that do not belong to the object with a specified color.

Parameters:

- `voxelarray ( np.ndarray )`: Boolean voxel array representing the object. Voxels with value `True` belong to the object, while voxels with value `False` are replaced by the specified color.
- `facecolor ( np.ndarray )`: 3D array of hexadecimal color strings representing the color assigned to each voxel.
- `color ( str , optional)`: Hexadecimal color string used for voxels outside the object. The string must start with `#` . Default is `'#ffffff'` .

Returns:

- `images ( list[np.ndarray] )`: List of RGB image arrays corresponding to the 2D slices of the masked object.

unalign_images(images, max_r, max_t, background_color='#ffffff')

Applies random misalignment transformations to a set of images trying to simulate alignment errors between images.

Parameters:

- `images ( np.ndarray )`: Array or list of RGB images to be randomly transformed.
- `max_r ( float )`: Maximum absolute rotation angle in radians. The applied rotation is randomly sampled between `-max_r` and `max_r`.
- `max_t ( float )`: Maximum translation value along the first translation axis. The applied translation is randomly sampled between `-max_t` and `max_t`.
- `background_color ( str , optional)`: Hexadecimal color string used as the padding background color. The string must start with `#`. Default is `'#ffffff'`.

Returns:

- `new_images ( list[np.ndarray] )`: List of transformed RGB images after applying random rotations and translations.

crop_images(images, max_c, sym=True, proba=1/5)

Randomly crops images by removing pixels from their borders.

Parameters:

- `images ( np.ndarray )`: Array or list of RGB images to be randomly cropped.
- `max_c ( list[int, int] )`: Maximum cropping values along the two image dimensions. The cropping amount is randomly selected between 1 and each corresponding maximum value.
- `sym ( bool , optional)`: Defines whether the cropping should be symmetric. If `True`, the same amount is removed from opposite sides of the image. If `False`, the cropping is randomly distributed between the two sides. Default is `True`.
- `proba ( float , optional)`: Probability that a given image will be cropped. Default is `1/5`.

Returns:

- `new_images ( list[np.ndarray] )`: List of images after applying random cropping. Images that are not selected for cropping are returned unchanged.

Image preparation

image_restoration(images, masks_path, masks_name, start=0, format='png', denoising=False, h=10, hColor=10, templateWindowSize=10, searchWindowSize=20)

Restores images by removing masked regions using image inpainting and optionally denoising.

Parameters:

- `images ( np.ndarray )`: Array or list of RGB images to be restored.
- `masks_path ( str )`: Path to the directory containing the image masks for inpainting.
- `masks_name ( str )`: Base name of the mask files for inpainting. Masks are expected to follow the naming format `masks_name_index.format`.
- `start ( int , optional)`: Starting index used when loading mask files. Default is `0`.
- `format ( str , optional)`: File format extension of the mask images. Default is `'png'`.
- `denoising ( bool , optional)`: If `True`, applies colored image denoising before inpainting. Default is `False`.
- `h ( int , optional)`: Filter strength for luminance components in the denoising algorithm. Default is `10`.
- `hColor ( int , optional)`: Filter strength for color components in the denoising algorithm. Default is `10`.
- `templateWindowSize ( int , optional)`: Size of the template patch used for denoising. Default is `10`.
- `searchWindowSize ( int , optional)`: Size of the window used to search for similar patches during denoising. Default is `20`.

Returns:

- `new_images ( list[np.ndarray] )`: List of restored RGB images after mask-based inpainting and optional denoising.

resize_images(images, background_color='#ffffff', add_padding=[0, 0])

Resizes a collection of images to a common size by adding padding.

Parameters:

- `images ( np.ndarray )`: Array or list of RGB images with potentially different dimensions.
- `background_color ( str , optional)`: Hexadecimal color string used to fill the added padding. The string must start with `#`. Default is `'#ffffff'`.
- `add_padding ( list[int, int] , optional)`: Additional padding added to the maximum image dimensions to avoid losing information during later alignment transformations. Default is `[0, 0]`.

Returns:

- `new_images ( list[np.ndarray] )`: List of resized RGB images with identical dimensions after padding.

Aligning images

`get_transfo(reference, to_align)`

Computes the rigid body transformation matrix required to align an image with a reference image, using the pyStackReg library.

Parameters:

- `reference ( np.ndarray )`: Reference image used as the alignment target.
- `to_align ( np.ndarray )`: Image to be aligned with the reference image.

Returns:

- `T ( np.ndarray )`: Transformation matrix representing the rigid body alignment between `to_align` and `reference`.

`aligned(images, ref='P')`

Aligns a sequence of images using image registration, using the pyStackReg library.

Parameters:

- `images ( list[np.ndarray] )`: List of RGB images to align.
- `ref ( str , optional)`: Reference strategy used for alignment. `'P'` aligns each image with the previous aligned image, while `'F'` aligns each image with the first image in the sequence. Default is `'P'`.

Returns:

- `aligned_im ( list[np.ndarray] )`: List of aligned RGB images. The first image is kept unchanged, and subsequent images are transformed according to their computed alignment matrices.
- `T_l ( list[np.ndarray] )`: List of transformation matrices used to align each image with its selected reference image.

`color_clustering(image, n_clusters)`

Groups the colors of an image into a specified number of clusters using the K-means clustering algorithm.

Parameters:

- `image ( np.ndarray )`: Image represented as an array of pixel color values. The image is converted to floating-point values before clustering.

- `n_clusters ( int )`: Number of color clusters to compute.

Returns:

- `labels ( list[int] )`: List of cluster labels assigned to each pixel in the input image.
- `centroids ( np.ndarray )`: Array containing the RGB values of the cluster centroids, represented as 8-bit unsigned integers.

color_classif(image, colors)

Assigns each pixel of an image to the closest reference color.

Parameters:

- `image ( np.ndarray )`: Image represented as an array of pixel RGB values.
- `colors ( np.ndarray )`: Array of reference RGB colors used for pixel classification.

Returns:

- `labels ( list[int] )`: List of indices identifying the closest reference color for each pixel in the input image.

create Voxels

rgb_to_facecolor(rgbcolor)

Converts an RGB image into a 2D array of hexadecimal color strings.

Parameters:

- `rgbcolor ( np.ndarray )`: RGB image represented as a 3D array of 8-bit unsigned integer pixel values.

Returns:

- `facecolor ( np.ndarray )`: 2D array of hexadecimal color strings, starting with `#`, where each element corresponds to the color of a pixel in the input RGB image.

merg_images(images)

Merges a collection of RGB images into a single array of pixel values.

Parameters:

- `images ( list[np.ndarray] )`: List of RGB images to merge. Each image is reshaped into a list of RGB pixel values before concatenation.

Returns:

- `glob_img ( np.ndarray )`: Two-dimensional array containing the RGB values of all pixels from the input images. Each row corresponds to one pixel, and each column corresponds to one color channel(R, G, B).

`unmerg_images(glob_img, nb_images, img_shape)`

Reconstructs a collection of RGB images from a merged array of pixel values.

Parameters:

- `glob_img ( np.ndarray )`: Two-dimensional array containing the RGB values of all pixels from the merged images.
- `nb_images ( int )`: Number of images to reconstruct from the merged pixel array.
- `img_shape ( tuple[int] )`: Shape of each reconstructed image, typically `(height, width, channels)`.

Returns:

- `images ( list[np.ndarray] )`: List of reconstructed RGB images with the specified shape.

`voxelize_images_fast(images, colors, background_index=0, path="")`

Converts a stack of 2D RGB images into 3D voxel objects by detecting connected components for each color using a 26-neighborhood structure, and saving the resulting voxel arrays.

Parameters:

- `images ( list[np.ndarray] )`: List of RGB images representing consecutive 2D slices of a 3D volume.
- `colors ( list[str] )`: List of hexadecimal color strings corresponding to the colors used to identify voxel objects.
- `background_index ( int , optional)`: Index of the background color in the `colors` list. This color is excluded from object detection. Default is `0`.
- `path ( str , optional)`: Directory path used to save the generated facecolor array and voxel object arrays as `.npy` files. Default is `''`.

Returns:

- `None` : This function does not return any value. The generated 3D facecolor array and connected voxel objects are saved as `.npy` files in the specified path.

Metrics

pixel_count(images)

Counts the occurrence of each color in a collection of RGB images.

Parameters:

- `images ( list[np.ndarray] )`: List of RGB images for which pixel color frequencies are computed.

Returns:

- `counts ( dict )`: Dictionary containing color occurrence information:
 - `colors ( np.ndarray )`: Array containing the unique hexadecimal color strings found across all images.
 - `total_counts ( np.ndarray )`: Array containing the total number of pixels associated with each color across the entire image set.
 - `per_image_counts ( list[np.ndarray] )`: List containing the pixel counts for each color in each individual image. Each element corresponds to one image and follows the same color ordering as `colors`.

color_distribution(counts)

Computes the color distribution from the pixel counts previously calculated by `pixel_count()`.

Parameters:

- `counts ( dict )`: Dictionary containing the pixel counts returned by `pixel_count()`. It must contain:
 - `colors ( np.ndarray )`: Array of unique colors found in the images.
 - `total_counts ( np.ndarray )`: Array containing the total number of pixels for each color across all images.
 - `per_image_counts ( list[np.ndarray] )`: List containing the number of pixels of each color for every individual image.

Returns:

- `dist ( dict )`: Dictionary containing the color distributions:
 - `colors ( np.ndarray )`: Array of unique colors considered in the distributions.
 - `global_distribution ( dict )`: Dictionary mapping each color to its proportion over all images combined.

- `per_image_distribution ( list[dict] )`: List of dictionaries, where each dictionary maps colors to their proportion within one individual image.

Analyses

`volume(voxelobject, voxel_dim=[1, 1, 1])`

Computes the volume of a voxel object taking into account the volume of a voxel.

Parameters:

- `voxelobject ( np.ndarray )`: Boolean voxel array representing the object. Voxels with value `True` are considered part of the object.
- `voxel_dim ( list[int, int, int] , optional)`: Physical dimensions of a voxel along the three axes, expressed as `[dx, dy, dz]`. Default is `[1, 1, 1]`.

Returns:

- `vol ( int )`: Volume of the voxel object, expressed in cubic units defined by `voxel_dim`.

`surface(voxelobject, voxel_dim=(1, 1, 1))`

Computes the surface area of a voxel object taking into account the dimensions of a voxel.

Parameters:

- `voxelobject ( np.ndarray )`: Boolean voxel array representing the object. Voxels with value `True` are considered part of the object.
- `voxel_dim ( list[int] , optional)`: Physical dimensions of a voxel along the three axes, expressed as `(dx, dy, dz)`. These dimensions are used to compute the area of each exposed face. Default is `(1, 1, 1)`.

Returns:

- `surface ( float )`: Surface area of the voxel object, expressed in square units defined by `voxel_dim`.

`bounding_box(voxelobject, voxel_dim=[1, 1, 1])`

Computes the axis-aligned bounding box of a voxel object.

Parameters:

- `voxelobject ( np.ndarray )`: Boolean or binary voxel array representing the object. Voxels with a non-zero or `True` value are considered occupied.
- `voxel_dim ( list[int] , optional)`: Physical dimensions of a voxel along the three axes, expressed as `[dx, dy, dz]`. These values are used to convert voxel indices into physical coordinates. Default is `[1, 1, 1]`.

Returns:

- `min_corner ( np.ndarray )`: Physical coordinates of the minimum corner of the bounding box, corresponding to the minimum occupied voxel indices.
- `max_corner ( np.ndarray )`: Physical coordinates of the maximum corner of the bounding box, corresponding to the boundary immediately after the maximum occupied voxel.
- `dimensions ( np.ndarray )`: Physical dimensions of the bounding box along the three axes, calculated as `max_corner - min_corner`.

Plots and images

`plot_voxels(voxelarray, facecolor, title="")`

Displays a 3D voxel object using Matplotlib's voxel visualization. The function renders all occupied voxels with the specified face colors and displays the result in a three-dimensional plot.

Parameters:

- `voxelarray ( np.ndarray )`: Boolean voxel array representing the object. Voxels with value `True` are displayed, while `False` voxels are ignored.
- `facecolor ( np.ndarray )`: Array of hexadecimal color strings specifying the color assigned to each voxel. It must have the same shape as `voxelarray`.
- `title ( str , optional)`: Title displayed above the figure. Default is `''`.

Returns:

- `None` : This function does not return any value. It displays a 3D voxel visualization.

`plot_images(images, labels, rows, cols, title="", axis1="", axis2="")`

Displays a collection of images arranged in a grid of subplots.

Parameters:

- `images ( list[np.ndarray] )`: List of images or plotting data to display. Images are displayed using `imshow()`, while plotting data are displayed as bar charts.
- `labels ( list[str] )`: List of titles associated with each subplot.
- `rows ( int )`: Number of rows in the subplot grid.
- `cols ( int )`: Number of columns in the subplot grid.
- `title ( str , optional)`: Title displayed above the figure. Default is `''`.
- `axis1 ( str , optional)`: Label for the horizontal axis of each subplot. Default is `''`.
- `axis2 ( str , optional)`: Label for the vertical axis of each subplot. Default is `''`.

Returns:

- `None` : This function does not return any value. It displays the figure containing the image grid.

save_slices(images, path, name)

Saves a collection of images as PNG files in a specified directory.

Parameters:

- `images ( list[np.ndarray] )`: List of images to save.
- `path ( str )`: Directory path where the image files will be saved.
- `name ( str )`: Base name used to generate the output filenames. Images are saved using the format `name_index.png`.

Returns:

- `None` : This function does not return any value. It saves the images as PNG files in the specified directory.

load_slices(filename, n, start=0, format='png')

Loads a sequence of image slices from disk and returns them as RGB images.

Parameters:

- `filename ( str )`: Base filename of the image sequence. Images are expected to follow the naming format `filename_index.format`.
- `n ( int )`: Number of images to load.
- `start ( int , optional)`: Starting index of the image sequence. Default is `0`.
- `format ( str , optional)`: File format extension of the images to load. Default is `'png'`.

Returns:

- `images ( list[np.ndarray] )`: List of loaded RGB images.

plot_clusters(data, labels, centroids, title="")

Displays the result of a clustering algorithm as a 2D scatter plot.

Parameters:

- `data ( np.ndarray )`: Two-dimensional array containing the coordinates of the data points to display. Each row corresponds to one observation, and the two columns represent its coordinates.
- `labels ( np.ndarray | list[int] )`: Cluster label assigned to each data point. The number of labels must match the number of rows in `data`.
- `centroids ( np.ndarray )`: Two-dimensional array containing the coordinates of the cluster centroids. Each row corresponds to the centroid of one cluster.
- `title ( str , optional)`: Title displayed above the figure. Default is `''`.

Returns:

- `None` : This function does not return any value. It displays a scatter plot of the clustered data and their corresponding centroids.

save_voxel_to_vtk(voxels, filename, voxel_dim=[1.0, 1.0, 1.0])

Saves a 3D voxel array as a VTK image data file (`.vtk`) that can be visualized and processed using software such as ParaView.

Parameters:

- `voxels ( np.ndarray )`: 3D boolean or binary voxel array representing the object. Occupied voxels should have a value of `True` or `1`, while empty voxels should have a value of `False` or `0`.
- `filename ( str )`: Path and filename of the output VTK file.
- `voxel_dim ( list[float, float, float] , optional)`: Physical dimensions of each voxel along the three axes, expressed as `[dx, dy, dz]`. These values define the spacing between voxels in the generated VTK file. Default is `[1.0, 1.0, 1.0]`.

Returns:

- `None` : This function does not return any value. It creates a VTK file containing the voxel data at the specified location.